

Konstruksi Perangkat Lunak

PERTEMUAN 3

Table-driven dan State-based Construction



Ekspektasi Luaran

Mampu menerapkan teknik table-driven dan state-based construction.



Outline

- Preface enum
 - Table-driven method
 - State-based method
- 

Table-driven dan State-based Construction

Preface: Enumerasi



Mengapa Enumerasi?

Jika anda diminta untuk membuat variable yang menyimpan jenis kelamin, tipe data apa yang akan anda gunakan?



Mengapa Enumerasi?

Menggunakan integer?

```
int jenisKelamin; //0 untuk Pria, 1 untuk Wanita
```

Atau menggunakan String?

```
String jenisKelamin; //"Pria" dan "Wanita"
```



Masalah dengan int dan String

- Integer dan String tidak menerapkan value checking di dalamnya.
`jenisKelamin = 5;`
- Pada integer sulit untuk mengingat nilai tiap kategori jika kategorinya banyak.
- Pada String sulit untuk memastikan bahwa ejaan tiap kategori tetap sama

```
jenisKelamin = "Laki-laki";  
jenisKelamin = "Pria";
```



Enumerasi

Enumerasi adalah struktur data yang memasangkan string yang menjadi kategori dengan integer yang menjadi indexnya.

```
enum JenisKelamin { PRIA, WANITA }
```

Kategori dapat diakses menggunakan syntax *namaEnum.kategori*

```
JenisKelamin jenisKelamin = JenisKelamin.WANITA;
```

Index dari kategori dapat digunakan dengan menggunakan casting ke integer

```
int indexJenisKelaminPria = (int)JenisKelamin.PRIA;
```



Enumerasi

Enumerasi sangat membantu saat kita menerapkan table-driven maupun state-based construction.

```
public enum State { State1, State2, State 3};  
public enum Index { Category1, Category2, Category3 };
```



Table-driven dan State-based Construction

Table-driven Construction Method



Driving Problem

Buatlah sebuah fungsi/method yang mengembalikan jumlah hari yang ada pada setiap bulan pada tahun non-kabisat!



Driving Problem

Seperti ini?

```
public class Kalender
{
    public enum Month { Januari, Februari, Maret, April, Mei,
        Juni, Juli, Agustus, September, Oktober, November, Desember }

    public static int GetDays(Month month)
    {
        if (month == Month.Januari)
            return 31;
        if (month == Month.Februari)
            return 28;
        if (month == Month.Maret)
            return 31;
        if (month == Month.April)
            return 30;
        if (month == Month.Mei)
            return 31;
        ...
    }
}
```

```
...
if (month == Month.Juni)
    return 30;
if (month == Month.Juli)
    return 31;
if (month == Month.Agustus)
    return 31;
if (month == Month.September)
    return 30;
if (month == Month.Oktober)
    return 31;
if (month == Month.November)
    return 30;
if (month == Month.Desember)
    return 31;
return 0;
}
}
```

Masalah Dengan Logic Statement

Apa yang menjadi masalah dengan kode tersebut?



Table-driven Construction

Penggunaan table (array) untuk melakukan pencarian nilai untuk mengganti penggunaan logic, terutama yang berukuran besar.



Table-driven Construction

Dengan masalah yang sama, codenya bisa sesederhana ini:

```
public class Kalender
{
    public enum Month { Januari, Februari, Maret, April, Mei, Juni, Juli, Agustus, September,
    Oktober, November, Desember }

    public static int GetDays(Month month)
    {
        int[] daysPerMonth = { 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31 };

        return daysPerMonth[(int) month];
    }
}
```

How to Table-driven Construction

1. Gunakan index untuk menggambarkan inputnya. (Disini menggunakan enum)

```
public enum Month { Januari, Februari, Maret, April, Mei, Juni, Juli, Agustus, September, Oktober, November, Desember }
```

Jika ada lebih dari satu input, buat sebanyak jumlah inputnya!



How to Table-driven Construction

2. Gunakan array untuk menggambarkan output sesuai dengan urutan indexnya.

```
int[] daysPerMonth = { 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31 };
```

Gunakan dimensi array sesuai dengan jumlah inputnya.



How to Table-driven Construction

3. Akses array output dengan menggunakan index yang sudah dibuat.

```
return daysPerMonth[(int) month];
```



Table-driven Construction

Perbandingan antara penggunaan **if** dan **table driven**:

Line of Code:

If: 40 Line

Table driven: 12 Line

Dengan 12 opsi, baris kode dapat berkurang hingga 25%nya.



Table-driven Construction

Positif	Negatif
• Kode yang dibuat lebih ringkas dan mudah dibaca. • Kode yang dibuat lebih mudah dipelihara.	• Penggunaan table akan menambah penggunaan memori. • Memelihara table dengan index yang banyak dapat menjadi sulit.



Table-driven dan State-based Construction

State-based Construction Method



State-based Construction

State-based programming, or automata-based programming, is a **programming technology using finite state machines to describe program behaviors.**



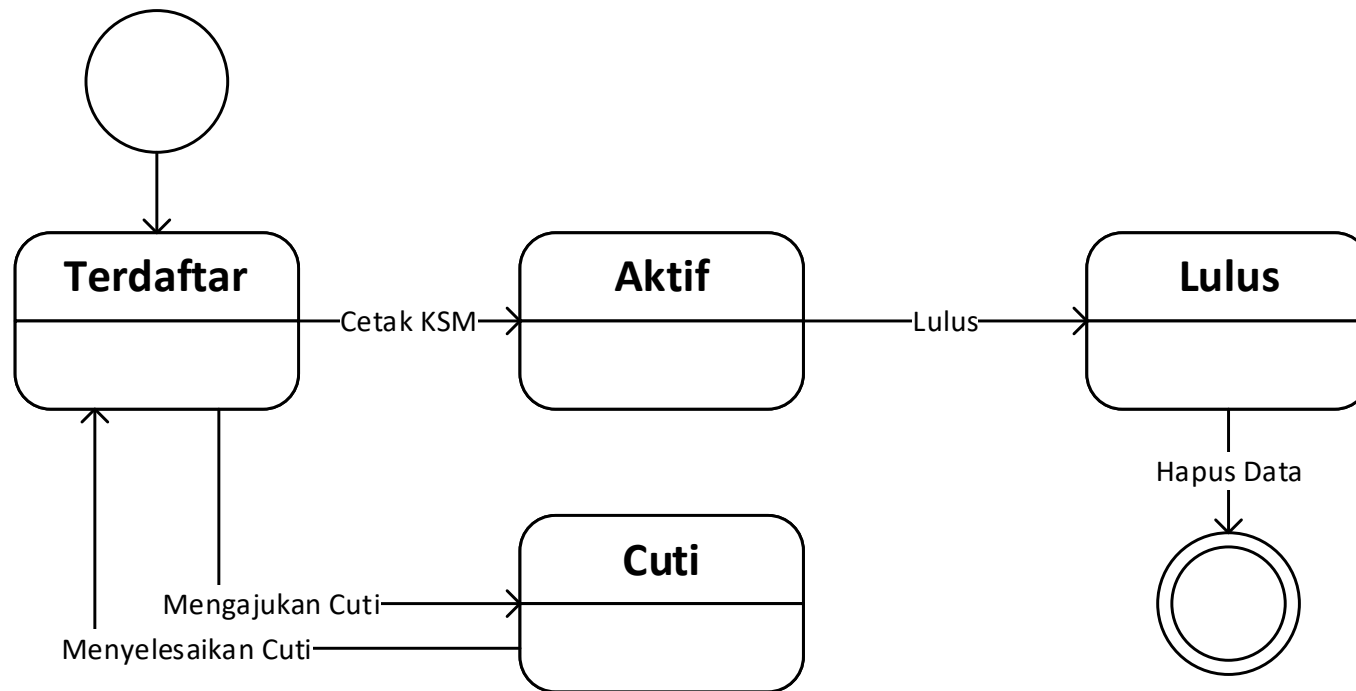
State-based Construction

Cara menerapkan automata:

1. Definisikan state dan transisinya.
 2. Buat struktur data yang mencatat state awal, transisi, dan state akhirnya.
 3. Petakan transisi antar state.
 4. Buat atribut yang menyimpan state saat ini.
 5. Set atribut ke state awal.
 6. Buat method/fungsi yang mencari state berikutnya.
 7. Buat method/fungsi yang mengganti state saat ini ke state berikutnya.
- 

Contoh

Status Mahasiswa



Contoh

Definisikan state dan transisinya.

```
public enum StudentState { TERDAFTAR, CUTI, AKTIF, LULUS};  
public enum Trigger { MENGAJUKAN_CUTI, MENYELESAIKAN_CUTI, CETAK_KSM, LULUS};
```

Disini kita menggunakan enum.



Contoh

Buat struktur data yang mencatat state awal, transisi, dan state akhirnya.

```
class transition
{
    public StudentState prevState;
    public StudentState nextState;
    public Trigger trigger;

    public transition(StudentState prevState, StudentState nextState, Trigger trigger)
    {
        this.prevState = prevState;
        this.nextState = nextState;
        this.trigger = trigger;
    }
}
```

Disini kita menggunakan class sebagai media struktur datanya. Dapat juga menggunakan struct atau lainnya.

Contoh

Petakan transisi antar state.

```
transition[] transitions =  
    {  
        new transition(StudentState.TERDAFTAR, StudentState.CUTI, Trigger.MENGAJUKAN_CUTI),  
        new transition(StudentState.TERDAFTAR, StudentState.AKTIF, Trigger.CETAK_KSM),  
        new transition(StudentState.CUTI, StudentState.TERDAFTAR, Trigger.MENYELESAIKAN_CUTI),  
        new transition(StudentState.AKTIF, StudentState.LULUS, Trigger.LULUS)  
    };
```

Disini kita menggunakan array, dapat juga menggunakan list atau sejenisnya.

Contoh

Buat atribut yang menyimpan state saat ini.

```
public StudentState currentState;
```



Contoh

Buat method/fungsi yang mencari state berikutnya.

```
public StudentState getNextState(StudentState prevState, Trigger trigger)
{
    StudentState nextState = prevState;

    for (int i = 0; i < transitions.Length; i++)
    {
        if (transitions[i].prevState == prevState && transitions[i].trigger == trigger)
            nextState = transitions[i].nextState;
    }

    return nextState;
}
```

Contoh

Buat method/fungsi yang mengganti state saat ini ke state berikutnya.

```
public void activateTrigger(Trigger trigger)
{
    StudentState nextState = getNextState(currentState, trigger);
    currentState = nextState;
}
```

Operasi Dalam State

Secara umum, operasi dalam state-based construction terjadi ketika sebuah transisi dilakukan.

```
public void activateTrigger(Trigger trigger)
{
    StudentState nextState = getNextState(currentState, trigger);
    currentState = nextState;

    Method1() //Method yang mengeksekusi operasi.
}
```

Q&A

Any questions or comments?



Further reading

- SWEBOOK V3.0

Table-driven Construction

- S. McConnell, Code Complete 2nd Edition, Microsoft Press, 1993.
Chapter 18

State-based Construction

- Tukkel N.I., Shalyto A.A. [State-based programming](#) // PC World. 2001.
#8, pp.116-121; #9, pp.132-138. (rus)
- 